SurgeonAI

AI Surgical Co-Pilot Glasses

VandyHacks XII | Build Reference Guide | v3.0

> *"A wearable AI surgical co-pilot that gives medical students, surgery residents, and trainees real-time procedural guidance through a glasses-mounted camera — grounded in verified medical protocol — with hands-free voice Q&A. Addresses the 250,000 preventable deaths caused by medical errors every year."*

Team roles

You — Backend	Python, OpenCV, all AI and API integrations, voice pipeline, hardware setup
Teammate — Frontend	React + Tailwind, surgical HUD interface, Flask video stream integration

Get these API keys before the hackathon starts

Groq	console.groq.com — free, no card required, takes 2 minutes
Gemini 2.5	aistudio.google.com — free tier, just a Google login
ElevenLabs	elevenlabs.io — free 10k chars/month, apply VandyHacks promo code
MongoDB	cloud.mongodb.com — free forever tier, no card
Snowflake	signup.snowflake.com — use your $400 student credits (120-day trial)

---

1. What We Are Building

SurgeonAI is a wearable AI system for surgical training. A Logitech Brio 100 webcam is mounted on the nose bridge of blue-light glasses, giving the system a first-person view of whatever the trainee is working on. The laptop runs a real-time AI pipeline that watches the camera feed, identifies what surgical step is in progress, detects mistakes, and speaks guidance out loud through ElevenLabs. The trainee can also ask questions completely hands-free using the Brio 100's built-in microphone — the system listens for a wake word, captures the question, pulls relevant surgical protocol from Snowflake, and speaks a grounded answer back within two seconds.

Three user modes

Mode	Who it's for	AI behavior	Key demo moment
Training	Med students	Step-by-step walkthrough, explains reasoning	Narrates entire procedure live
Assist	Surgery residents	Silent until mistake, then intervenes immediately	Catches wrong instrument live
Q&A	All trainees	Answers spoken questions completely hands-free	Resident asks aloud, AI responds

What makes this different

- The AI is not guessing — Snowflake Cortex grounds every response in real surgical protocol documents.
- The system sees the surgical field — Gemini 2.5 Flash is a Vision Language Model that processes actual camera frames, not just text descriptions.
- It is completely hands-free — voice input from the glasses microphone means the trainee never looks away from the patient.
- It is wearable — the glasses are the interface. No screen to look at, no keyboard to touch.

2. Full Tech Stack

Hardware

Component	Item	Purpose	Cost
Camera + Mic	Logitech Brio 100	Webcam on glasses for surgeon POV. Built-in mic array captures spoken questions hands-free.	~$25 Best Buy
Glasses	Blue-light glasses (yours)	Frame to mount the camera on. Already have.	Free
Mount	Electrical tape + foam pad	Secure webcam to nose bridge. Foam from paper towel. Tape from Walmart.	~$2
Display	Your laptop (fullscreen)	Runs everything. Shows the live surgical HUD to judges.	Already have
Audio out	Laptop speakers	ElevenLabs voice guidance plays through these.	Already have

AI & APIs

Service	Model	Role in the system	Cost
Gemini 2.5	Flash Preview	Vision Language Model. Receives camera frames every 3 seconds. Identifies surgical step, detects tool errors, describes scene.	Free
Groq	Llama 3.3 70B	Text generation engine. Receives Gemini scene description + Snowflake context. Produces clinical guidance text in under 200ms.	Free
ElevenLabs	Turbo v2	Converts Groq text to human-sounding clinical voice. Plays through laptop speakers. Rachel voice — calm and clinical.	Free — 10k chars/month
Snowflake	Cortex LLM	RAG layer. Stores surgical procedure documentation. Searched before every Groq call to ground responses in real medical protocol.	Free — $400 credits
MongoDB	Atlas free tier	Session logging. Records every step, warning, and Q&A exchange to a database for the session history feature.	Free forever

> *Snowflake $400 credits: a hackathon will use roughly $2-5 of Snowflake compute. Your $400 is effectively unlimited for the weekend — use Cortex freely for RAG queries and LLM calls without watching the balance.*

Python libraries — backend

Library	Install command	What it does
opencv-python	pip install opencv-python	Reads webcam frames, performs color detection, draws HUD overlays on video
google-generativeai	pip install google-generativeai	Gemini 2.5 Flash API — sends frames and prompts, receives scene analysis JSON
groq	pip install groq	Groq API — sends text prompts, receives surgical guidance in under 200ms

elevenlabs	pip install elevenlabs	ElevenLabs API — converts text to audio, plays clinical voice through speakers
SpeechRecognition	pip install SpeechRecognition	Captures audio from Brio 100 mic, transcribes spoken questions
flask + flask-socketio	pip install flask flask-socketio	Serves MJPEG video stream and pushes AI events to the React frontend
pymongo	pip install pymongo	Writes session events to MongoDB Atlas
snowflake-connector-python	pip install snowflake-connector-python	Queries Snowflake Cortex for surgical protocol context
python-dotenv	pip install python-dotenv	Loads API keys from .env file — never hardcode keys
pyttsx3	pip install pyttsx3	Offline TTS fallback in case ElevenLabs quota is reached

> *Install everything at once: pip install opencv-python google-generativeai groq elevenlabs SpeechRecognition flask flask-socketio pymongo snowflake-connector-python python-dotenv pyttsx3*

Frontend stack — teammate owns this

Tool	Purpose
React + Vite	Main app framework. Fast dev server, easy component structure.
Tailwind CSS	Utility CSS. Build the surgical HUD design quickly without writing custom CSS.
Socket.IO client	Receives real-time AI events from the Flask backend: step changes, warnings, Q&A updates.
MJPEG stream	Display the live annotated video feed from Python directly in the browser.

3. How the System Works

Two independent loops run in parallel at all times. Loop A is the vision loop — it watches the camera, understands the scene, and delivers guidance. Loop B is the voice loop — it listens for questions and answers them. Both loops share the same state object so they stay in sync.

Loop A — Vision guidance (every 3 seconds)

Step	What happens	Tool used
1	OpenCV reads a frame from the Brio 100 webcam	OpenCV VideoCapture
2	Frame is encoded as a JPEG image in memory	OpenCV imencode
3	Frame is sent to Gemini 2.5 Flash with a surgical analysis prompt	Gemini VLM API
4	Gemini returns structured JSON: current step, tool correct or not, warning, instruction	Gemini response
5	Step tracker advances if Gemini confirms a new step has been reached	Python state machine
6	Snowflake Cortex is queried for protocol relevant to the current step and scene	Snowflake Cortex
7	Groq generates concise guidance text using Gemini output + Snowflake context	Groq Llama 3.3
8	ElevenLabs speaks the guidance text through the laptop speakers	ElevenLabs Turbo v2
9	OpenCV draws the HUD overlay on the frame — step counter, warning banner, highlights	OpenCV drawing functions
10	Socket.IO pushes the step and warning state to the React frontend	Flask-SocketIO
11	MongoDB logs the full event — step, warning, instruction, timestamp	pymongo

Loop B — Hands-free voice Q&A (always running)

Step	What happens	Tool used
1	SpeechRecognition listens continuously via the Brio 100 built-in microphone	SpeechRecognition library
2	Wake word 'Hey Surgeon' is detected in the audio stream	String matching on transcript
3	The resident's question is captured in the next audio segment	recognizer.listen()
4	Snowflake Cortex searches surgical guidelines for context relevant to the question	Snowflake Cortex search
5	Groq generates a grounded answer using: the question, Snowflake context, current step	Groq Llama 3.3

6	ElevenLabs speaks the answer through the laptop speakers	ElevenLabs Turbo v2
7	The Q&A pair is pushed to the React frontend to display in the HUD panel	Flask-SocketIO
8	The Q&A exchange is logged to MongoDB with a timestamp	pymongo

Why Gemini AND Groq — not just one

Gemini 2.5 Flash (VLM)	Groq + Llama 3.3
Receives camera frame as an image	Receives text only — no image
Understands what is physically in the scene	Generates fast, natural clinical language
Identifies step, tools, errors from visual input	Produces spoken guidance and Q&A answers
Runs every 3 seconds — scene understanding	Runs on every detection event — under 200ms

4. Hardware Setup

Mounting the webcam on glasses

Do this before writing a single line of code. Get it stable first, then build around it.

1. Remove or fold down the monitor clip from the bottom of the Brio 100 — you don't need it.
2. Fold a paper towel three times into a small square pad. Place it on the nose bridge of the glasses as a cushion and grip layer.
3. Set the webcam body on top of the foam pad, lens pointing forward.
4. Plug the USB cable into your laptop and open the camera feed first — before any tape goes on.
5. Adjust the angle until what you see on the laptop screen matches exactly what you are looking at. This alignment is critical for the demo.
6. Once aligned, run two strips of electrical tape horizontally across the top and bottom of the camera body, wrapping under the glasses frame.
7. Run two more strips vertically on each side. This cross pattern prevents wobble in all four directions.
8. Tape the USB cable along the temple arm of the glasses every few centimetres so it does not pull on the camera.

> *Camera index tip: the Brio 100 may show up as device index 0, 1, or 2 depending on how many cameras your laptop has. Test by running a quick Python script that tries VideoCapture(0), VideoCapture(1) until you see the correct feed.*

Finding the Brio 100 microphone

The Brio 100 has a built-in dual microphone array. It shows up as a separate audio input device on your laptop. You need to find its device index before writing the voice Q&A code.

- Open Python and print the list of available microphone device names from the SpeechRecognition library.
- Look for 'Brio' or 'USB Audio' in the list — note its index number.
- Set that index in your .env file as BRIO_MIC_INDEX before starting the voice loop.

---

5. Demo Surgery Setup

Props to prepare

Banana	The patient. Yellow skin gives good contrast for the red marker lines.
Red permanent marker	Draw 2-3 lines across the banana skin — these represent arteries and the incision zone.
Red tape on handle	Wrap a plastic utensil handle in red tape — this is the wrong instrument.
Blue tape on handle	Wrap another utensil in blue tape — this is the scalpel (correct for step 2).
Green tape on handle	Wrap a third utensil in green tape — this is the clamp (correct for step 3).
Tray or board	Operating surface. Keeps props in a fixed position relative to the camera.
Red food coloring	Optional. Small dish nearby. Adds realism if you want it for dramatic effect.

Color detection map

OpenCV detects these colors in the camera frame using HSV masking. Each color triggers a specific AI response.

What OpenCV sees	What it means	AI response triggered
Red lines on banana skin	Artery present / incision zone	Highlights zone on HUD, voice confirms zone identified, enters step 1

Blue tape near red zone	Correct instrument — scalpel	Confirms correct tool, voice says 'scalpel confirmed', advances to next step
Red tape near red zone	Wrong instrument	Warning fires: 'Incorrect instrument detected — select scalpel'
Green tape near red zone	Clamp — correct for step 3	Confirms clamp, voice acknowledges, advances to close step

The 4-step demo procedure

Walk judges through this exact sequence. Target time: 90 seconds. Rehearse it until it is smooth.

9. **Identify:** Camera sees the banana. Gemini reads the red marker lines. Voice says: *"Incision zone identified at marked region. Select scalpel to proceed."*

10. **Wrong tool:** Pick up the red-tape instrument. System fires immediately: *"Incorrect instrument detected. Scalpel required for this incision."* **Pause. Let it land.**

11. **Correct tool:** Put down wrong tool, pick up blue-tape scalpel. Voice confirms: *"Scalpel confirmed. Make shallow incision along the marked zone. Maintain 2cm clearance."*

12. **Voice Q&A:** Say out loud: *"Hey Surgeon, what is the safe depth for this incision?"* **System answers within 2 seconds. Judges see it is completely hands-free. This is the moment.**

---

6. Frontend Spec — For the Teammate

> *This section is written for the frontend developer. The HUD is what judges see for 90 seconds straight. It matters as much as the AI. Make it look like medical OS software, not a web project.*

What to build

A fullscreen surgical HUD web app running in the browser. It receives a live annotated video stream from the Python backend and overlays real-time AI status information on top of it. The video is the background. Everything else floats over it.

Design rules — non-negotiable

- Background: pure black (#000000). No grays, no gradients, no transparency on the outer container.
- Safe state color: clinical green (#1D9E75). Used for confirmed actions, correct tool, step complete.
- Warning color: amber (#EF9F27). Used for cautions and advisories.
- Critical / error color: red (#E24B4A). Used for wrong instrument and critical warnings.
- Font: JetBrains Mono or Courier New for all HUD text. Monospace reads as technical and clinical.
- Borders: thin 1px only. No box shadows. No rounded corners on status components. Sharp edges feel medical.
- No animations except a single pulsing border on the warning banner when active.

HUD layout — four zones

Zone	Screen position	Content	Updates when
Video feed	Full background layer	Live MJPEG stream from Flask at localhost:5000/video	Continuous — always playing
Step tracker	Top left overlay	Current step number, step name, simple progress indicator (step X of 4)	Gemini confirms new step reached
Status banner	Top right overlay	GREEN: Procedure nominal / RED: Warning text from AI	Any AI detection event fires
Q&A panel	Bottom overlay	Last question asked + AI answer, fades after 10 seconds	Voice Q&A wake word triggered

Data flow from backend to frontend

The Python backend communicates with React through two channels:

- **Video stream:** Flask serves an MJPEG stream at /video. React renders it in a full-background img tag. No WebRTC, no complexity — just a URL in an img src.
- **AI events:** Flask-SocketIO emits three event types to the React frontend in real time: step_update (current step data), warning (warning text), and qa_update (question + answer pair). React listens for these and updates component state.

What the backend emits

step_update	Fires when Gemini confirms the step has advanced. Payload: step number and step name. React updates the step tracker component.

warning	Fires when a mistake is detected. Payload: warning text string. React turns the status banner red and displays the text.
qa_update	Fires when voice Q&A completes. Payload: question string and answer string. React displays both in the Q&A panel.
clear	Fires when a warning is resolved. React returns status banner to green 'Procedure nominal'.

7. Snowflake Setup

Snowflake is used as a RAG (Retrieval-Augmented Generation) layer. You store surgical procedure documentation in it, then query it before every Groq call so the AI's guidance is grounded in real medical protocol rather than model knowledge alone.

What to store in Snowflake

- Step-by-step procedure guides for the procedures you are simulating (incision, clamping, closure).
- Safe depth and angle parameters for each procedure step.
- Instrument specifications — which tool is correct at each stage and why.
- Complication protocols — what to do if bleeding is detected, if the wrong area is cut, etc.
- You do not need real medical textbooks. Write simplified reference documents that describe your demo scenario. Snowflake Cortex will search them accurately.

How the RAG flow works

13. Before calling Groq, the backend sends the current question or scene description to Snowflake Cortex.
14. Cortex searches your stored documents using semantic similarity — it finds the most relevant protocol paragraphs.
15. The retrieved context is injected into the Groq prompt alongside the scene description.
16. Groq's response is now grounded in your stored medical data, not just model inference.
17. This is the line that wins the Patient Safety prize: 'Our AI does not hallucinate surgical advice. Every response is grounded in verified protocol stored in Snowflake.'

Snowflake Cortex functions used

CORTEX.COMPLETE	Calls an LLM (Llama 3, Mistral, etc.) directly from a SQL query. Used for generating answers from retrieved context.

CORTEX.SEARCH	Semantic search over your stored documents. Used for finding relevant protocol before each Groq call.

> *You have $400 in credits. The entire hackathon will cost under $5 in Snowflake compute. Run Cortex freely — do not worry about the balance.*

---

8. Hour-by-Hour Execution Plan

> *Color guide: Backend = you. Frontend = teammate. Both = work together. Times are estimates. If something takes longer than expected, protect the core demo above everything else.*

Phase 1 — Environment setup (Hours 1–2)

#	Task	Who	Time
1	Sign up for all API keys: Groq, Gemini, ElevenLabs, MongoDB, Snowflake	Both	30 min
2	Create .env file with all keys. Install all pip packages. Verify imports.	Backend	20 min
3	Mount webcam on glasses: foam pad, electrical tape cross pattern, verify angle	Backend	20 min
4	Open OpenCV, confirm webcam feed displays correctly on laptop screen	Backend	10 min
5	Find Brio 100 mic device index using SpeechRecognition microphone list	Backend	10 min
6	Create React + Vite app, install Tailwind, set black background, basic layout	Frontend	25 min
7	Draw red marker lines on banana, prepare colored tape on three plastic utensils	Both	10 min

Phase 2 — Core backend AI pipeline (Hours 3–10)

#	Task	Who	Time
8	OpenCV main loop: read frame from Brio 100, detect red, blue, green by HSV color	Backend	45 min

9	Draw basic HUD overlay on frame: step label, colored bounding boxes, warning text	Backend	40 min
10	Gemini integration: encode frame, send surgical prompt, parse JSON response	Backend	60 min
11	Step state machine: Python dict of 4 steps, advance based on Gemini output	Backend	40 min
12	Snowflake: create table, insert surgical protocol documents, test Cortex query	Backend	45 min
13	Groq integration: receive scene + Snowflake context, generate guidance text	Backend	30 min
14	ElevenLabs integration: convert Groq output to audio, play in background thread	Backend	30 min
15	Flask MJPEG stream: serve annotated OpenCV frames at /video endpoint	Backend	30 min
16	Flask-SocketIO: emit step_update, warning, qa_update events from Python	Backend	25 min
17	MongoDB: log every AI event to Atlas on each loop iteration	Backend	20 min

Phase 3 — Frontend HUD (Hours 4–12, runs in parallel)

#	Task	Who	Time
18	Display MJPEG stream from Flask as full-background video in React	Frontend	30 min
19	Build step tracker: step number, step name, X of 4 indicator — top left	Frontend	45 min
20	Build status banner: green 'Nominal' state and red 'Warning: [text]' state	Frontend	30 min
21	Build Q&A panel: question + answer display, bottom of screen, auto-fades	Frontend	35 min
22	Wire Socket.IO: connect to Flask, handle step_update, warning, qa_update events	Frontend	40 min
23	Polish: pure black background, monospace font, sharp borders, no shadows	Frontend	60 min
24	Test frontend with backend running — verify all three event types display correctly	Both	30 min

Phase 4 — Voice Q&A pipeline (Hours 11–14)

#	Task	Who	Time
25	Verify Brio 100 mic records clearly — test in a quiet area of the venue	Backend	10 min
26	Implement always-on listener: wake word 'Hey Surgeon' triggers listen mode	Backend	30 min
27	Wire Q&A pipeline: question → Snowflake context → Groq answer → ElevenLabs speak	Backend	40 min
28	Emit qa_update to frontend so Q&A panel updates in real time	Backend	15 min
29	End-to-end test: ask a question out loud, verify full flow completes under 2 seconds	Both	20 min

Phase 5 — Prize integrations (Hours 14–16)

#	Task	Who	Time
30	Verify MongoDB Atlas is receiving events — check dashboard shows session data	Backend	15 min
31	Verify Snowflake is being queried — check query history in Snowflake console	Backend	10 min
32	GitHub: create repo, write detailed README, push all code with clean commit history	Both	30 min
33	Confirm ElevenLabs promo code from VandyHacks is applied to your account	Backend	5 min
34	Confirm Gemini API key is from aistudio.google.com and linked to your Google account	Backend	5 min

Phase 6 — Polish and demo prep (Hours 17–22)

#	Task	Who	Time
35	Full end-to-end demo run: glasses on, banana on table, run all 4 steps + voice Q&A	Both	30 min
36	Tune Gemini prompt until step detection is reliable and consistent on banana setup	Backend	40 min

37	Tune ElevenLabs: confirm Rachel voice sounds right, adjust speed if needed	Backend	15 min
38	Final HUD polish: every pixel should look intentional and medical	Frontend	60 min
39	Cable management: tape USB cable cleanly along glasses arm, no loose wires	Backend	15 min
40	Rehearse full demo 8 times. Target 90 seconds. No stumbles.	Both	60 min
41	Sleep.	Both	Mandatory

Phase 7 — Submission (Hours 23–24)

#	Task	Who	Time
42	Submit to: 1st Overall and Patient Safety $500 prize track	Both	10 min
43	Submit to: ElevenLabs and Gemini API sponsor tracks	Both	5 min
44	Submit to: MongoDB Atlas, Snowflake, and GitHub sponsor tracks	Both	5 min
45	Set up expo table: glasses displayed, HUD running fullscreen, props arranged neatly	Both	10 min
46	Final rehearsal at expo table with full hardware rig before judging starts	Both	15 min

9. Prize Strategy

Submit to every applicable track. Most require zero additional work — you are already using the technology.

Prize	Track	Why your project qualifies	Extra work required
1st Overall	Overall judging	Wearable AI, real medical problem, physical hardware, insane demo, voice interaction	0 — this is your core build
Patient Safety $500	Sponsor track	250k deaths stat + AI preventing surgical errors = this prize was written for you	0 — just submit and mention the stat in your pitch

ElevenLabs — earbuds	Sponsor track	Voice is not a feature — it is the core output mechanism of the entire system	0 — already integrated, just submit
Gemini API — swag	Sponsor track	Gemini is the brain of your scene understanding — it is the most critical AI component	0 — already integrated, just submit
MongoDB — IoT Kit	Sponsor track	Every step, warning, and Q&A is logged to Atlas — real persistence with real data	25 min — already built
Snowflake — Pi 4	Sponsor track	Cortex RAG grounds every AI response in verified medical protocol	45 min — already built
GitHub — swag	Sponsor track	Clean repo, detailed README, meaningful commit history showing real collaboration	25 min — just keep repo tidy
Most Creative	Overall judging	Nobody else will have wearable glasses with vision AI and hands-free voice Q&A	0 — just demo it well

> *Do not spend any time on Solana. Blockchain adds zero value to a surgical AI project and forcing it would waste hours and confuse judges.*

---

10. Pitch Script

Memorize this. 90 seconds maximum. Every word is deliberate. The pauses are as important as the words.

```
[Put on the glasses. Say nothing. Let judges look at the hardware for 3 seconds.]

"Medical errors kill over 250,000 Americans every year."

"Not because surgeons are careless. Because they are human."

[pause 2 seconds]

"This is SurgeonAI. It watches what a surgical trainee sees, understands what they are doing, and guides every step in real time — grounded in verified medical protocol."

[pick up the wrong instrument near the banana]

"Watch what happens when I reach for the wrong tool."

[ElevenLabs fires: "Incorrect instrument detected. Scalpel required for this incision."]

[pause — say nothing for 3 full seconds. Let it hit.]

"It caught that before any harm was done."
```

```
[say out loud: 'Hey Surgeon, what is the safe incision depth here?']
[AI answers within 2 seconds]
"Completely hands-free. The resident never looks away from the patient."
"Gemini sees the field. Snowflake grounds it in real protocol. Groq generates the guidance. ElevenLabs speaks it. Under two seconds."
[take off glasses. make eye contact with judges. stop talking.]
```

11. Critical Reminders

Tonight — before the hackathon

- Sign up for all five API keys: Groq, Gemini, ElevenLabs, MongoDB, Snowflake
- Install all pip packages on your laptop and verify imports work without errors
- Apply the ElevenLabs promo code from the VandyHacks XII registration email
- Buy electrical tape at Walmart — $2, takes 5 minutes
- Charge your laptop fully
- Download Ollama locally as a backup in case hackathon WiFi is unreliable

First 30 minutes at the hackathon

- Mount the webcam on glasses before writing any code — get hardware stable first
- Verify the camera feed opens correctly in Python before touching any API
- Find the Brio 100 mic device index and record it in your .env file
- Draw red marker lines on the banana immediately so the ink has time to dry
- Confirm all five API keys authenticate successfully — do this before starting the build

During the build

- Backend: get Gemini successfully reading the banana scene before touching Groq or ElevenLabs
- Frontend: visual quality of the HUD matters more than adding extra features — prioritize polish
- Both: run a full demo rehearsal every 4 hours — catch integration issues before they compound
- If any integration breaks and costs more than 1 hour to fix — skip it and protect the core demo
- Never commit API keys to GitHub — use .env files and add .env to .gitignore

At the expo table

- Glasses on before judges walk up — not during the pitch
- Open with the 250,000 deaths statistic. Every single time. No exceptions.
- Never say 'prototype.' Never say 'we ran out of time.' Never apologize for anything.
- When ElevenLabs speaks — stop talking completely. Let the silence hit. That is your best moment.
- When the voice Q&A fires and the AI answers — pause after the answer. Let judges absorb it.
- If something breaks mid-demo: stay calm, say 'let me show you the core feature' and pivot to what works

Built in 24 hours.

Could save lives in 24 months.

Good luck. Go win.